

Pose Estimation Project

Roy Levy

Ron Shavit

1. General Implementation

The project is an interactive system for estimating the position and orientation of a camera inside a 3D terrain.

The terrain is created from a height-map image and rendered using OpenGL. The system estimates the camera pose from correspondences between 2D image points and known 3D world points.

Three methods are implemented for creating these correspondences:

- Manual selection of matching 2D and 3D points.
- Automatic detection of colored trackers.
- Automatic feature matching using SIFT, FLANN, and ASIFT-style augmentation.

The main application is implemented in `world_split_v5.22.py`.

This file manages the window, camera movement, rendering, program modes, user input, and pose estimation.

2. Terrain and Rendering

The file `image_to_tris.py` converts a grayscale height map into a 3D triangle mesh.

The function `sample_pixels` samples pixels from the image at fixed intervals. Each pixel brightness value is converted into a terrain height.

The function `build_triangles` connects neighboring sampled points. Each group of four neighboring vertices is divided into two triangles. The result is saved in a `.tri` file.

The file `trimap_beta.py` reads the terrain file. The function `read_tri_map` creates the terrain vertices and triangle objects from the stored data.

In the main file, the function `build_terrain_vbo` creates an OpenGL Vertex Buffer Object containing the terrain vertices and colors. The function `draw_terrain_vbo` draws the terrain using `glDrawArrays`.

Using a VBO improves the rendering performance because the terrain data is sent to the graphics card once instead of being rebuilt during every frame.

The program window is created using Pygame and is divided into two OpenGL viewports. The function `draw` renders both views. The function `render_scene` handles camera movement, keyboard input, drawing, and switching between the different program modes.

3. Camera Pose Estimation

3.1. Manual Picking

Manual Picking Mode is activated using **P**.

The user selects a point on the 3D terrain and then selects its matching location in the 2D image.

The function `get_world_coords` reads the depth value of the selected pixel and uses `gluUnProject` to convert the mouse position into a 3D world coordinate.

After at least four 2D–3D correspondences are created, the function `solve_pnp` estimates the camera pose.

The implementation first uses `solvePnP` with the EPNP method. RANSAC is used to reduce the effect of incorrect correspondences. If necessary, the system also tries a direct PnP solution and improves the result using `solvePnPRefineLM`.

The program calculates the reprojection error, camera position error, and camera rotation error. It also renders the estimated camera view as a transparent overlay on the real view.

3.2. Colored Trackers

Trackers Mode is activated using **T**.

The tracker positions and colors are stored in `trackers.trk`. The function `get_trackers_from_file`, implemented in `trackers.py`, reads the 3D position and RGB color of every tracker.

When the user presses **B**, the program captures the current camera view. The function `find_blob_centers`, implemented in `color_picking.py`, searches the image for the known tracker colors.

A color mask is created for each tracker. Connected-component detection is used to find the colored regions, and the center of mass of each region is used as its 2D image position.

The 2D center is matched with the known 3D tracker position. These correspondences are passed to the function `solve_pnp_trackers`, which estimates the camera position and orientation.

The real and estimated camera poses are saved and displayed in the 3D environment for comparison.

3.3. Feature Matching

Feature Matching Mode is activated using **F**.

Before running this mode, the feature database is prepared using:

```
python world_split_v5.22.py -pre
```

In Feature Pre Mode, the function `create_feature_reference_view` renders a reference image and detects SIFT keypoints and descriptors.

The user connects selected SIFT keypoints to matching 3D terrain points. These mappings are stored in the feature database.

During the normal Feature Matching Mode, the program captures the current camera image and detects SIFT descriptors.

The function `match_query_to_database` compares the descriptors in the current image with the descriptors stored in the database.

The comparison is performed using the FLANN matcher. For every query descriptor, the two nearest database descriptors are found. Lowe’s ratio test is then used to reject weak or unclear matches.

The accepted matches create automatic 2D–3D correspondences. These correspondences are passed to the function `solve_pnp_feature`, which uses RANSAC and PnP to estimate the camera pose.

The quality of the result is evaluated using the number of inliers, the inlier ratio, and the reprojection error.

4. Feature Database and ASIFT

The file `feature_database.py` contains the classes `FeatureDatabase` and `ReferenceView`.

A reference view stores its camera pose, SIFT keypoints, SIFT descriptors, creation time, and mappings between keypoints and 3D points.

The function `add_reference_view` adds a new rendered reference image to the database. The function `add_mapping` connects a selected keypoint to a 3D world coordinate.

The database is saved as a compressed NPZ file. It also contains metadata about the terrain map, image size, view resolution, and feature settings. The function `validate_metadata` checks that a loaded database matches the currently selected map and configuration.

The project also contains ASIFT-style augmentation. Its purpose is to improve feature matching when the camera viewpoint is very different from the reference viewpoint.

The function `simulate_asift_view` applies rotations, blurring, and horizontal scaling to create simulated views of a reference image.

The function `augment_reference_view_with_asift` detects new SIFT descriptors in these simulated images and connects them to already known 3D points. This allows the database to store several descriptor variants for the same 3D location.

5. Main Libraries

Pygame is used for creating the application window, receiving keyboard and mouse input, and managing the main program loop.

PyOpenGL is used for rendering the terrain, controlling the viewports, managing depth and transparency, and working with projection and camera matrices.

OpenCV is used for SIFT feature detection, FLANN feature matching, image processing, `solvePnP`, `solvePnP_Ransac`, `solvePnPRefineLM`, and point projection.

NumPy is used for vectors, matrices, descriptors, coordinate conversions, error calculations, and NPZ database files.

SciPy is used for connected-component processing and calculating the centers of the colored tracker regions.

Pillow is used for loading and processing images during tracker detection.

6. Running the Project

The dependencies are installed using:

```
pip install -r requirements.txt
```

The main program is started using:

```
python world_split_v5.22.py
```

Feature Pre Mode is started using:

```
python world_split_v5.22.py --pre
```

7. GitHub Repository

The complete source code is available in the GitHub Repository:

<https://github.com/RonShavit/PoseEstimationProject>